

Derived, Not Authoritative: Pattern B as the Substrate-Derived View Position in the CKS Composition Model

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 5, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, in operational form, the second of the three composition patterns the CKS hybrid-composition specification supports — *adjacent component as derived view of substrate content* (Pattern B) — as a standalone architectural commitment that can be defended, implemented, and tested independently of the other two patterns and the broader composition framework.

Abstract

The CKS hybrid composition specification names three positions an adjacent AI component can occupy relative to a CKS substrate: input to a cell (Pattern A), derived view of substrate content (Pattern B), or separate concern (Pattern C). This note formalizes Pattern B as a standalone architectural specification. Pattern B is the architectural pattern by which a substrate's content is indexed, embedded, summarized, or projected into an adjacent component — a vector index for semantic search, a derived knowledge graph, a search-optimized projection, a summary store — to support specific access paths without that component becoming authoritative for coordination. The note states Pattern B's four operational components, distinguishes Pattern B from four adjacent variations commonly conflated with it, names ten failure modes that violate the pattern, and supplies an operational test (with three sharpening properties from the foundational note) for whether a derived-view boundary satisfies Pattern B. The most common failure is gradual: a derived view becomes faster or more convenient than the substrate it derives from, then becomes the surface participants reach for first, then becomes the place new content effectively lives. Pattern B's four operational components specifically preempt that drift.

1. Why Pattern B needs to be formalized as standalone

The parent foundational note (A1.16) commits to three composition patterns. The integrating-frame note (A2.91) names Pattern B at the integrating level, and the sibling specialization A2.92 formalizes Pattern A. This note formalizes Pattern B as having independent architectural content, with particular weight on the source-of-truth-migration failure that distinguishes Pattern B violations from coherent Pattern B uses.

Three motivations make the standalone treatment load-bearing. *First*, Pattern B's commitments are distinct from Patterns A and C. Pattern A places the adjacent component inside a cell's reasoning

environment; Pattern C places it outside coordination scope entirely; Pattern B places it as a projection of substrate content — a position that interacts with coordination state through the content it derives from without being authoritative for coordination decisions. The interaction is what makes the position consequential and its independent formalization load-bearing. *Second*, Pattern B carries strategic prior-art weight: vector indexes, derived knowledge graphs, and search-optimized projections are the dominant operational pattern for hybrid AI deployments in 2024–2026, and patentable derivations focused on derived-view layers, performance-optimized projections, or vector-indexed AI systems are more defensibly contested when Pattern B is publicly formalized as a standalone commitment. *Third*, Pattern B operationalizes source-of-truth at the derived-view boundary: the substrate's authority for "what is the case" (A2.43) and "what is current" (A2.44) is not self-enforcing when adjacent components carry derived projections, and without an explicit pattern, source-of-truth can be compromised by views becoming authoritative through use patterns. The pattern operationalizes those commitments at the derived-view boundary while respecting the linear-cost commitment (A1.06) that bounds update mechanisms.

2. Pattern B, defined precisely

Pattern B is the architectural pattern in which a substrate's content is indexed, embedded, or summarized into an adjacent component to support specific queries. Specifically: (a) the substrate's content is the source for the view's generation — the view is built from substrate content through indexing, embedding, summarization, or projection, operations that produce alternative access paths to substrate content without altering it; (b) the view is read as a derived view — reads against it return content that originated in substrate, and the view is operationally a projection rather than an independent store; (c) the view is not authoritative — when conflicts arise between the view's state and the substrate's state, the substrate is the reference, and the view may lag substrate, may have stale content during update windows, or may produce approximate results without any of these conditions shifting authority to the view; and (d) writes flow to substrate, not to the view — cells writing coordination state write to substrate per Pattern A's requirements, with the view updated downstream rather than as a primary write path.

Pattern B is the architectural pattern by which substrate's content is made available through alternative access paths without the access path becoming authoritative for coordination.

3. The four operational components

The architectural commitment is articulated as four operational components, each of which must hold for a derived-view boundary to satisfy Pattern B.

Component (a) — Substrate as source for view generation. The view's content originates in substrate per Guarantee C of the determinism contract (A2.59) — substrate provides the addressable source from which the view is derived. Content not present in substrate cannot legitimately appear in the view, except as transformations (embeddings, summaries, indexes) of content that is.

Component (b) — View regeneratable from substrate state. The view's content must be derivable from substrate at any time. If substrate content changes, the view is updated; if the view drifts, the substrate is the reference and the view is rebuilt. A view that has accumulated content the substrate cannot regenerate has stopped being a derived view and become an independent store the deployment must

now govern as substrate, which is not the position the architecture supports. Regeneratability is architectural, not configurational — it must hold by design, not by retention policy. A view that nominally permits regeneration but has accumulated state regeneration would discard is not regeneratable in the architectural sense.

Component (c) — View non-authoritative on coordination questions. Reads from the view that affect coordination decisions must be reconciled against substrate before being treated as authoritative. The substrate is authoritative for "what is the case" (A2.43) and "what is current" (A2.44). The view is for performance or convenience — semantic search, summarization, or faster lookup along access paths the substrate's native shape does not optimize for. Correctness on coordination questions comes from substrate, not from the projection.

Component (d) — Writes do not flow back to the view. When a cell writes coordination state, it writes to substrate; the view is updated downstream. A pattern in which cells write to the view and substrate is reconstructed from the view inverts the source-of-truth commitment, and the architecture does not permit that inversion. If a deployment finds itself writing to the view because substrate's interface is inconvenient, the substrate's interface needs work; the source of truth does not migrate. This component preserves AI-as-substrate-mediator (A1.04), specifically Property B per A2.20 (LLM writes under orchestration rules), at the derived-view boundary by ensuring no substrate writes flow from the view.

A derived-view boundary that satisfies all four components has Pattern B in the architectural sense.

4. What Pattern B does not claim

The pattern does not claim that all substrate-content access must go through derived views; direct substrate reads per A2.59 remain operationally fundamental, and views are convenience layers, not required access paths. It does not foreclose multiple derived views of the same substrate content; a deployment may carry several Pattern B views supporting different access paths, each evaluated against the four components per its own boundary. It does not require views to update synchronously with substrate writes; views may have update latency, since A2.44 specifies that substrate is the reference for current state, not that views must match substrate instantly. It does not specify implementation patterns for view generation; batch indexing, streaming updates, periodic regeneration, differential updates, or any combination are admissible so long as the four components hold. It does not foreclose performance optimization through derived views — that is the pattern's purpose — provided the optimization does not migrate authority. It does not require views to be read-only at the operational layer; deployments may permit operational manipulation (pinning entries, cached results) provided no coordination-state writes flow through the view as a primary path.

5. What Pattern B is not

Four adjacent variations are commonly conflated with Pattern B; each fails one or more of the four components.

Not vector-index-as-substrate-substitute. Architectures in which a vector index operates as the system's substrate — coordination state lives in the vector index rather than in CKS substrate, with the index treated as authoritative — fail component (c). Vector indexes in Pattern B are derived from substrate; they are not the substrate.

Not write-through-cache patterns. Patterns where writes go to both the cache and the underlying store simultaneously, with reads served from the cache, fail component (d). Writes in Pattern B go to substrate first; the view is updated downstream, not in parallel as a write target.

Not bidirectional-sync architectures. Architectures in which substrate and adjacent component update each other through automated processes fail component (b). When both stores write to each other, neither is unambiguously the source. Pattern B is unidirectional — substrate to view. (A2.95 develops this anti-pattern as "hidden bidirectional coupling.")

Not derived-view-becomes-authoritative-by-default. Architectures in which the view is operationally treated as authoritative because it is faster or more convenient than substrate fail component (c). Pattern B's non-authoritativeness is architectural; views remain non-authoritative regardless of operational convenience or performance advantages.

6. Why Pattern B is load-bearing for downstream commitments

Pattern B is one of three positions in the integrating hybrid-composition specification (A1.16, A2.91); without it, the architecture would have no pattern for performance and convenience layers over substrate content. It operationalizes Category 1 source-of-truth (A2.43) at the derived-view boundary by committing to non-authoritativeness on "what is the case" questions, and preserves Category 2 source-of-truth (A2.44) by routing coordination-state writes through substrate. It preserves AI-as-substrate-mediator (A1.04) — specifically Property B per A2.20 — because writes flow to substrate through cells, not to the view. It maintains the substrate-as-source-of-truth decomposition (A2.42–A2.48) at the view boundary. It respects linear-cost scaling (A1.06): view update mechanisms must be chosen to preserve A1.06, and a view whose update cost grows superlinearly with substrate size is a deployment concern rather than an architectural relaxation Pattern B grants. It preserves path retraceability (A1.07): the retraceable trail lives in substrate, and views — which produce no substrate writes — do not extend the trail directly. It preserves tool-agnosticism (A1.05): views can be implemented in various tools without affecting substrate's host environment.

7. Failure modes that violate Pattern B

Ten failure modes name ways an implementation can claim Pattern B nominally while failing it operationally. Each shifts the source of truth out of substrate or accumulates content the substrate cannot regenerate.

(a) View accumulates content substrate cannot regenerate. The view contains content that did not originate from substrate and cannot be reconstructed from substrate state; component (b) fails and the view has stopped being derived.

(b) Cells write to the view directly. Cells write coordination state to the view (e.g., a vector index that cells write embeddings to directly) with substrate updated only as a downstream step or not at all; component (d) fails.

(c) Source of truth migrates through use patterns. The deployment starts with substrate as authoritative and the view as derived, but over time the view becomes the surface participants reach for first, then the place new content effectively lives, until substrate is operationally a backup or audit trail; component (c) fails through accumulated drift even when architecturally claimed otherwise. This is the failure

mode §6.2 of the source paper names as *context rot* — substrate content participants thought authoritative has been compressed, summarized, or re-embedded out of fidelity with what the substrate carries.

(d) *View used as authoritative without substrate reconciliation.* Cells read from the view and treat its content as authoritative for coordination decisions without reconciling against substrate; component (c) fails for specific decision-making paths.

(e) *View rebuilt rarely or never.* The view is technically regeneratable but operationally never rebuilt; it has drifted from substrate without being reconciled, and component (b) fails operationally even when nominally satisfied.

(f) *Bidirectional update flow.* Substrate and view update each other through automated processes — substrate writes propagate to view, and view writes propagate to substrate, with neither direction passing through a cell or rule; the substrate–cell boundary (A1.02) is silently bypassed and component (d) fails.

(g) *View governance treated as substrate governance.* Governance interfaces (inspection, modification, override) operate on the view rather than on substrate; humans exercising governance per A1.01 may modify the view without modifying substrate, and the view becomes the governance target.

(h) *View selection-bias becomes substrate selection-bias.* The view has selective coverage of substrate content (e.g., a vector index covering some categories but not others), and operations using the view treat the covered subset as the entirety of substrate content; the view's partial coverage shifts the operational representation of substrate.

(i) *Performance optimization erodes regeneratability.* Performance optimizations accumulate in the view (caching of computed values, pre-processed aggregations) that substrate cannot reproduce; component (b) fails because the performance gains have created content the substrate does not contain.

(j) *Multiple views drift relative to each other.* Multiple views of substrate content drift not only from substrate but from each other, with no single reference; components (a) and (b) fail in combination.

The ten failure modes share a common shape: an adjacent component that began as a derived view has acquired content, write authority, or operational primacy the architecture does not grant.

8. Operational test

A derived-view boundary satisfies Pattern B if and only if all of the following are true at all times during the deployment's existence:

1. Substrate is the source for view generation per component (a).
2. The view is regeneratable from substrate state per component (b) — at any time, the view can be rebuilt from substrate state without information loss.
3. The view is non-authoritative on coordination questions per component (c) — when the view and substrate disagree, the substrate wins.
4. Writes to coordination state flow to substrate per component (d), with the view updated downstream.

5. Three sharpening properties (from §4 of the foundational note A1.16) hold operationally, not just nominally: regeneratability is enforced operationally rather than merely permitted; substrate reconciliation occurs before coordination decisions are treated as authoritative; and writes do not happen to the view as a primary path, with substrate as the write target and view updates downstream.
6. The four components hold for every derived view in the deployment, operationally rather than nominally — the architectural commitment is per-view Pattern B satisfaction.

A derived-view boundary that fails any of (1)–(6) does not satisfy Pattern B in the architectural sense, regardless of how the adjacent component is described in deployment documentation.

9. The one-sentence test

If an adjacent component contains content derived from substrate, can be regenerated from substrate state at any time, is treated as non-authoritative when reconciled against substrate, and does not receive coordination-state writes as a primary path, the component is in Pattern B; if any of these properties is missing, the component has acquired a position the architecture does not name. The four-component specification in §3 and the three sharpening properties in §8 supply the full architectural definition for cases requiring detailed analysis.

10. Why naming Pattern B as standalone matters

Implementations under pressure to deliver hybrid AI architectures with performance optimizations consistently drift toward derived views that gradually become authoritative through use patterns. The drift is steady rather than catastrophic: derived views are operationally attractive because they are faster or more convenient than the substrate they derive from; commercial AI products typically have caching and indexing layers without explicit non-authoritativeness commitments; and audiences understand "the system has a search index" more readily than "the derived view is non-authoritative; substrate remains the source of truth even when the view is operationally faster."

Implementations that drift away from Pattern B produce systems where the source of truth migrates from substrate to derived view through accumulated use. The downstream consequences manifest as substrate-authority erosion, context-rot failures per §6.2, governance failures (humans exercising governance over the view rather than over substrate), and the obscuring of the architectural commitments A2.43 and A2.44 defend.

Naming Pattern B as a standalone architectural commitment — with the four operational components in §3, the limits in §4, the four adjacent-variation distinctions in §5, the load-bearing connections in §6, the ten failure modes in §7, the operational test in §8, and the one-sentence test in §9 — gives downstream readers a precise specification of what derived-view hybrid compositions must satisfy. Sibling notes A2.94 and A2.95 specialize Pattern C and the anti-patterns respectively; together with A2.92 (Pattern A) they close the decomposition of A1.16.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Derived, Not Authoritative: Pattern B as the Substrate-Derived View Position in the CKS Composition Model*. May 5, 2026. ORCID: 0009-0004-8065-3235.